Lecture 07

Object-Oriented Programming

Polymorphism, Abstraction

Prepared by **Hok Tin**

Content

- Polymorphism
  - Down-casting
  - Up-casting
- Abstraction
- Swing cont.

Polymorphism

- The ability of ***an object to take on many forms***. The most common use of polymorphism in OOP occurs when a parent class reference is used to refer to a child class object.
- A concept by which we can perform a single action in different ways.
- It is derived from 2 Greek words: poly and morphs. The word "**poly**" means **many** and "**morphs**" means **forms**. So, polymorphism means many forms.

```
1  class A {}
2  class B extends A {}
3
4  B obj = new B();
```

(Normal)

```
1  class A {}
2  class B extends A {}
3
4  A obj = new B();
```

Parent class A created an object with its own subclass.

Overloading Method

❑ Example #1

```
1  class Bank {
2      int getRateOfInterest() {
3          return 0;
4      }
5  }
6
7  class ABA extends Bank {
8      int getRateOfInterest() {
9          return 8;
10     }
11 }
12
13 class ACILEDA extends Bank {
14     int getRateOfInterest() {
15         return 7;
16     }
17 }
18 class CANADIA extends Bank {
19     int getRateOfInterest() {
20         return 9;
21     }
22 }
```

(Normal)

```
24 class MainTest {
25     public static void main(String args[]) {
26         ABA aba = new ABA();
27         ACILEDA aci = new ACILEDA();
28         CANADIA cana = new CANADIA();
29         System.out.println("ABA Rate of Interest: " + aba.getRateOfInterest());
30         System.out.println("ACILEDA Rate of Interest: " + aci.getRateOfInterest());
31         System.out.println("CANADIA Rate of Interest: " + cana.getRateOfInterest());
32     }
33 }
```

Polymorphism

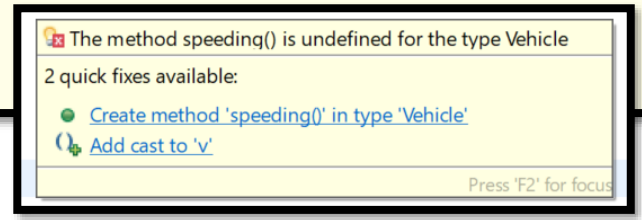
```
1  class TestPolymorphism {
2      public static void main(String args[]) {
3          Bank b;
4          b = new ABA();
5          System.out.println("ABA Rate of Interest: " + b.getRateOfInterest());
6          b = new ACILEDA();
7          System.out.println("ACILEDA Rate of Interest: " + b.getRateOfInterest());
8          b = new CANADIA();
9          System.out.println("CANADIA Rate of Interest: " + b.getRateOfInterest());
10     }
11 }
```

Overloading Method

❑ Example #2: casting any override methods

```
class Vehicle{
    void moving() {
        System.out.println("Vehicle is moving...");
    }
}

class Bike extends Vehicle {
    void moving() {
        System.out.println("Bike is moving...");
    }
    void speeding() {
        System.out.println("Running at speed 60km/h");
    }
}

public class Demo {
    public static void main(String[] args) {
        Vehicle v = new Bike();
        v.moving();
    }
}
```

```
public class Demo {
    public static void main(String[] args) {
        Vehicle v = new Bike();
        v.moving();
        v.speeding();
    }
}
```



Output:

```
Bike is moving...
```

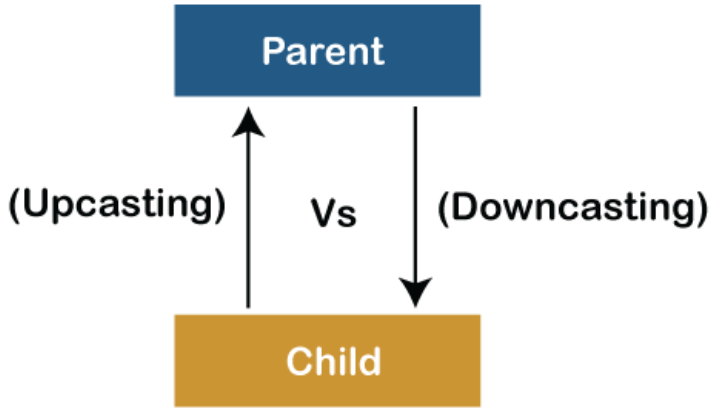
Overloading Method

Up-casting Vs Down-casting

A process of converting one data type to another is known as ***Typecasting*** but ***Up-casting*** and ***Down-casting*** are the type of object typecasting

- **Parent** and **Child** objects are two types of objects. So, there are two types of typecasting possible for an object
- ***Parent to Child*** and ***Child to Parent*** or can say Upcasting and Downcasting
- We use Upcasting when we need to develop a code that ***deals with only the parent class***
- *Downcasting is used when we need to develop a code that* ***accesses behaviors of the child class***

```
1  class A {}
2  class B extends A{}
3
4  A a = (A) new B();
```




```
1  class A {}
2  class B extends A{}
3
4  A a = new B();
5  B b = (B)a;
```

Overloading Method

Up-casting Vs Down-casting

```java
class  Parent {
    void PrintData() {
        System.out.println("method of parent class");
    }
}

class Child extends Parent {
    void PrintData() {
        System.out.println("method of child class");
    }
}
class Example {
    public static void main(String args[]) {
        Parent obj1 = (Parent) new Child();
        Parent obj2 = (Parent) new Child();
        obj1.PrintData();
        obj2.PrintData();
    }
}
```

Output:

```
Method of child class
Method of child class
```

```java
class Parent {
    String name;
    void showMessage() {
        System.out.println("Parent method is called");
    }
}

class Child extends Parent {
    int age;
    void showMessage() {
        System.out.println("Child method is called");
    }
}

public class Example {
    public static void main(String[] args) {
        Parent p = new Child();
        p.name = "Tola";

        Child c = (Child)p;

        c.age = 18;
        System.out.println(c.name);
        System.out.println(c.age);
        c.showMessage();
    }
}
```

Output:

```
Tola
18
Child method is called
```

Abstraction

A class which is declared with the abstract keyword is known as an ***abstract class*** in Java. It can have abstract and non-abstract methods (method with the body).

- Abstract classes ***may*** or ***may not contain abstract methods***
- But, if a class has ***at least one abstract method***, then the class must be ***declared abstract***
- If a class is declared abstract, it cannot be instantiated.
- To use an abstract class, you ***have to inherit*** it from another class, ***provide implementations*** to the abstract methods in it.
- If you inherit an abstract class, you have to ***provide implementations to all the abstract methods*** in it.

❑ **Abstract class**

```
abstract class A {}
```

```
abstract class A {
    abstract void methodName();
}
```

Abstraction

- **Example:** of Abstract class that has an abstract method

```
abstract class Bike {
    abstract void run();
}

class Honda extends Bike {

}
```

The type Honda must implement the inherited abstract method Bike.run()

2 quick fixes available:

- Add unimplemented methods
- Make type 'Honda' abstract

Press 'F2' for focus

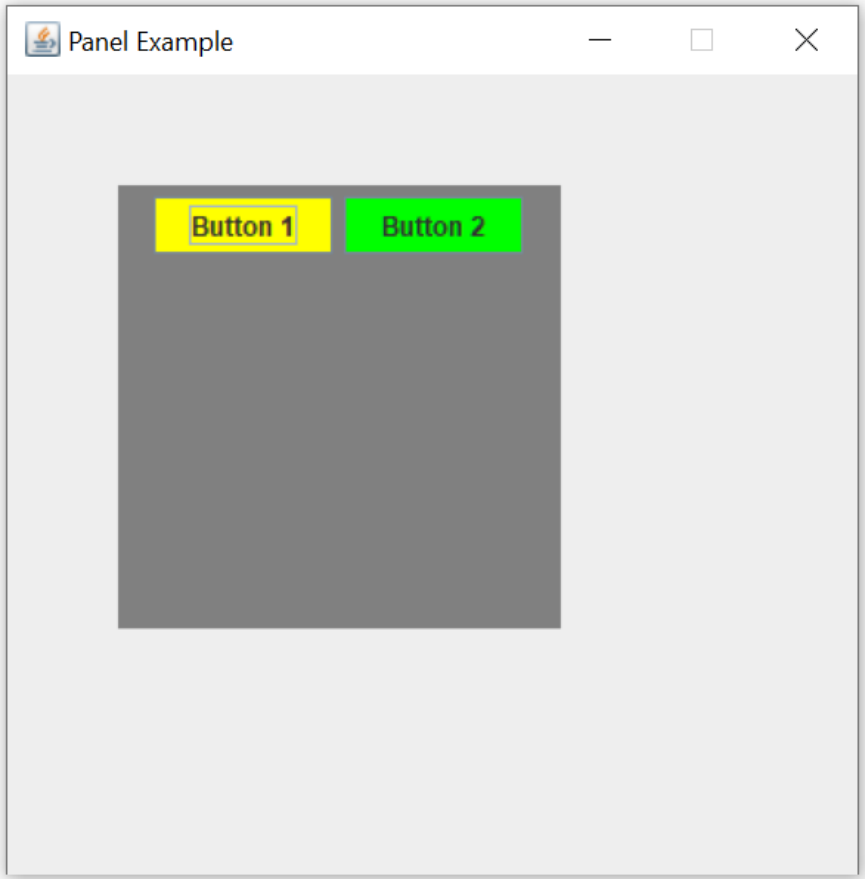
```
abstract class Bike {
    abstract void run();
}

class Honda extends Bike {

    @Override
    void run() {
        System.out.println("Running safely");
    }
}

class Demo {
    public static void main(String args[]) {
        Honda obj = new Honda();
        obj.run();
    }
}
```

Swing Components

❑ JPandel

- Is a simplest container class
- It provides space in which an application can attach any other component

```
public class Demo {
    public static void main(String args[]) {
        JFrame f = new JFrame("Panel Example");

        JPanel panel = new JPanel();
        panel.setBounds(50, 50, 200, 200);
        panel.setBackground(Color.gray);

        JButton b1 = new JButton("Button 1");
        b1.setBounds(50, 100, 80, 30);
        b1.setBackground(Color.yellow);

        JButton b2 = new JButton("Button 2");
        b2.setBounds(100, 100, 80, 30);
        b2.setBackground(Color.green);

        panel.add(b1);
        panel.add(b2);

        f.add(panel);

        f.setSize(400, 400);
        f.setLayout(null);
        f.setResizable(false);
        f.setVisible(true);
    }
}
```

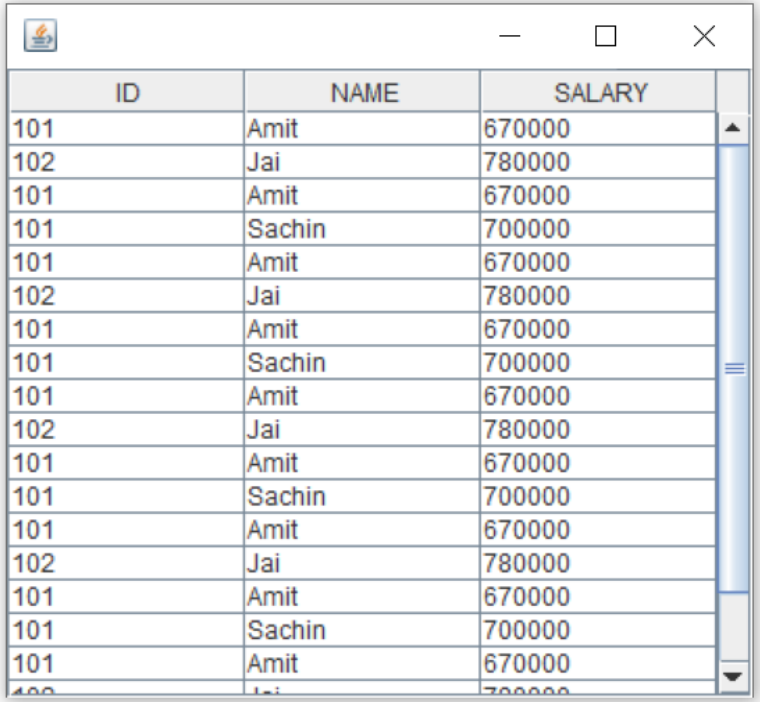


Swing Components

❑ JTable

- Is used to display data in tabular form
- It is composed of rows and columns

```
public class Demo {
    public static void main(String args[]) {

        JFrame f = new JFrame();

        String data[][] =
            {
                { "101", "Amit", "670000" },
                { "102", "Jai", "780000" },
                { "101", "Amit", "670000" },
                { "101", "Sachin", "700000" }
            };

        String column[] = { "ID", "NAME", "SALARY" };

        JTable jt = new JTable(data, column);

        jt.setBounds(30, 40, 200, 300);

        JScrollPane sp = new JScrollPane(jt);
        f.add(sp);

        f.setSize(300, 400);
        f.setVisible(true);
    }
}
```

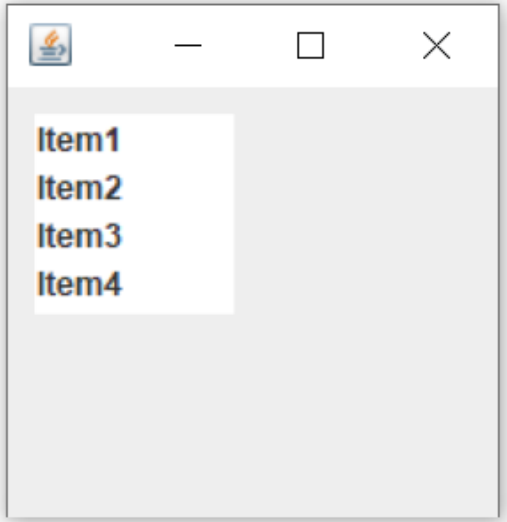


Swing Components

JList

- Represents a list of text items
- It is composed of rows and columns

```
public class Demo {
    public static void main(String args[]) {
        JFrame f = new JFrame();

        DefaultListModel<String> l1 = new DefaultListModel<>();
        l1.addElement("Item1");
        l1.addElement("Item2");
        l1.addElement("Item3");
        l1.addElement("Item4");

        JList<String> list = new JList<>(l1);
        list.setBounds(100, 100, 75, 75);

        f.add(list);
        f.setSize(400, 400);
        f.setLayout(null);
        f.setVisible(true);
    }
}
```

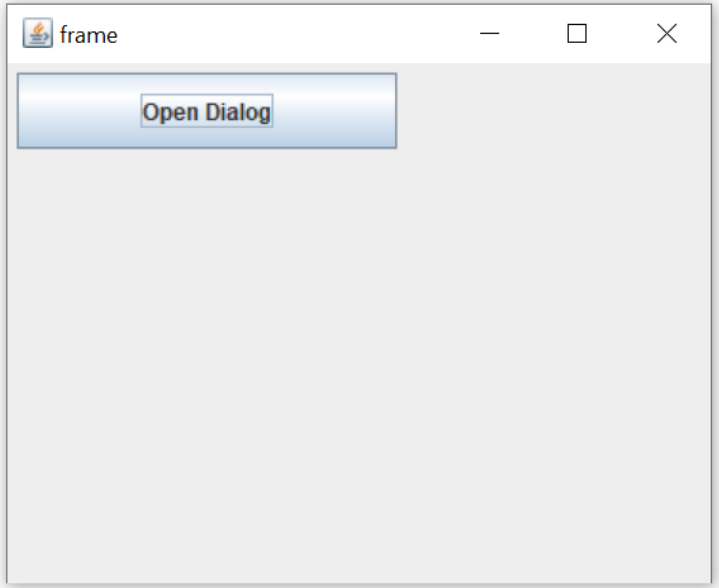


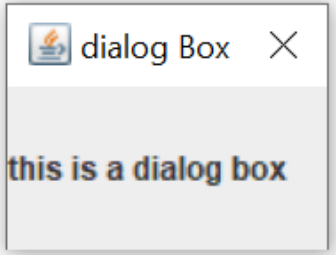
Swing Components

JDialog

- Represents a top-level window with a border and a title used to take some form of input from the user

```
public class Demo {
    public static void main(String args[]) {
        JFrame f = new JFrame("frame");
        JButton b = new JButton("Open Dialog");
        b.setActionCommand("click");
        b.setBounds(5, 5, 200, 40);
        f.add(b);

        JDialog d = new JDialog(f, "dialog Box");
        JLabel l = new JLabel("this is a dialog box");
        d.add(l);
        d.setSize(100, 100);

        b.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String s = e.getActionCommand();
                if (s.equals("click")) {
                    d.setVisible(true);
                }
            }
        });

        f.setSize(400, 400);
        f.setLayout(null);
        f.setVisible(true);
    }
}
```





Swing Layouts

GridLayout

- is used to arrange the components in a rectangular grid.
- One component is displayed in each rectangle

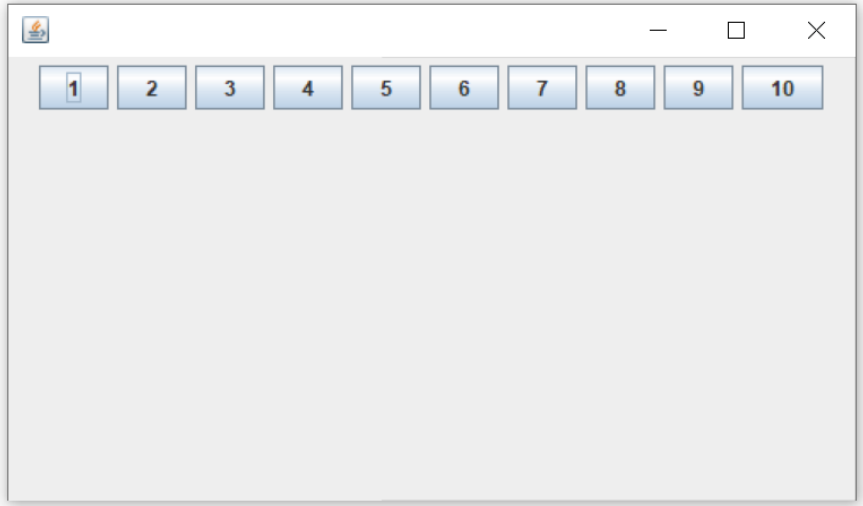
```
public class Demo {
    public static void main(String args[]) {
        JFrame f = new JFrame();
        JButton b1 = new JButton("1");
        JButton b2 = new JButton("2");
        JButton b3 = new JButton("3");
        JButton b4 = new JButton("4");
        JButton b5 = new JButton("5");
        JButton b6 = new JButton("6");
        JButton b7 = new JButton("7");
        JButton b8 = new JButton("8");
        JButton b9 = new JButton("9");

        // adding buttons to the frame
        f.add(b1);
        f.add(b2);
        f.add(b3);
        f.add(b4);
        f.add(b5);
        f.add(b6);
        f.add(b7);
        f.add(b8);
        f.add(b9);

        // setting grid layout of 3 rows and 3 columns
        f.setLayout(new GridLayout(3, 3));
        f.setSize(300, 300);
        f.setVisible(true);
    }
}
```

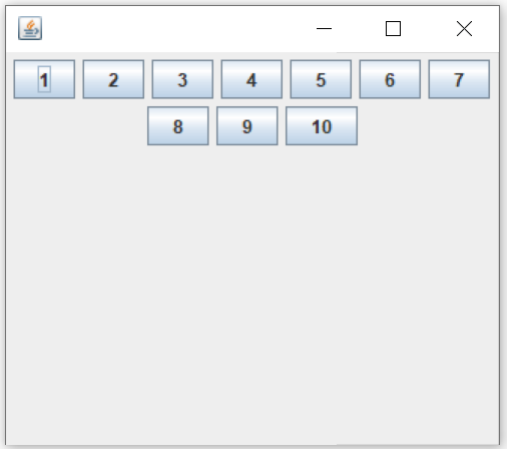
Swing Layouts

FlowLayout

- is used to arrange the components in a line, one after another (in a flow)

```
public class Demo {
    public static void main(String args[]) {
        // creating a frame object
        JFrame f = new JFrame();

        // creating the buttons
        JButton b1 = new JButton("1");
        JButton b2 = new JButton("2");
        JButton b3 = new JButton("3");
        JButton b4 = new JButton("4");
        JButton b5 = new JButton("5");
        JButton b6 = new JButton("6");
        JButton b7 = new JButton("7");
        JButton b8 = new JButton("8");
        JButton b9 = new JButton("9");
        JButton b10 = new JButton("10");

        // adding the buttons to frame
        f.add(b1);
        f.add(b2);
        f.add(b3);
        f.add(b4);
        f.add(b5);
        f.add(b6);
        f.add(b7);
        f.add(b8);
        f.add(b9);
        f.add(b10);

        // as default, alignment is center
        f.setLayout(new FlowLayout());

        f.setSize(300, 300);
        f.setVisible(true);
    }
}
```



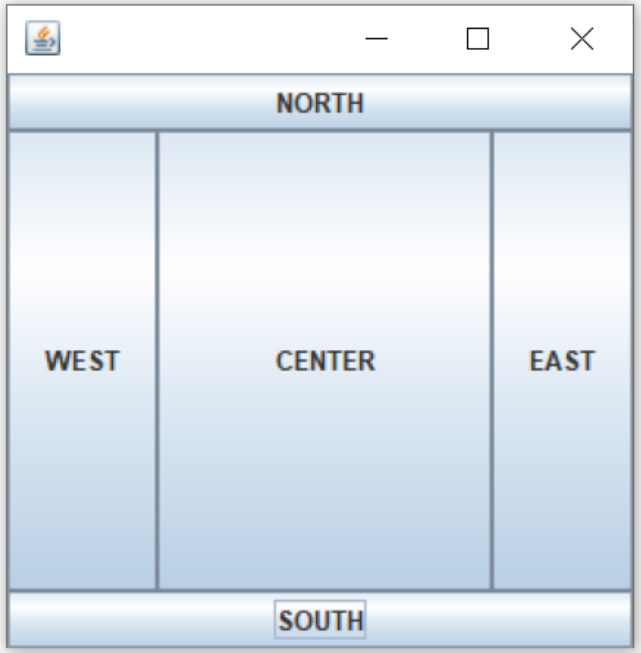


Swing Layouts

BorderLayout

- is used to arrange the components in five regions: north, south, east, west, and center

```
public class Demo {
    public static void main(String args[]) {
        JFrame f = new JFrame();

        // creating buttons
        JButton b1 = new JButton("NORTH");
        JButton b2 = new JButton("SOUTH");
        JButton b3 = new JButton("EAST");
        JButton b4 = new JButton("WEST");
        JButton b5 = new JButton("CENTER");

        f.add(b1, BorderLayout.NORTH);
        f.add(b2, BorderLayout.SOUTH);
        f.add(b3, BorderLayout.EAST);
        f.add(b4, BorderLayout.WEST);
        f.add(b5, BorderLayout.CENTER);

        f.setSize(300, 300);
        f.setVisible(true);
    }
}
```



Good luck 👌

😉 **References:**

https://www.javatpoint.com/java-swing

https://www.javatpoint.com/java-layout-manager